

חלק 2 - קורס תכנות מונחה עצמים C++

מדוע בחרנו בפרויקט Cinematic

בחרנו בפרויקט Cinematic משום שלדעתנו הוא מציג בצורה טובה יותר את עקרונות התכנות המונחה עצמים שנלמדו בקורס, הן מבחינת מבנה המחלקות והן מבחינת הקשרים ביניהן.

הפרויקט כולל מספר היררכיות של **הורשה**. לדוגמה, המחלקה Person משמשת כמחלקת בסיס עבור Guest ו- Employee, ובהמשך Guest מורשת ל- Customer ול- Reviewer. בנוסף קיימת היררכיית אולמות הכוללת את Hall, VIPHall, 3DHall ו- DVIPHall3, וכן היררכיית כרטיסים הכוללת את Ticket ו- VIPTicket. מבנה זה מאפשר הרחבה נוחה של המערכת והוספת סוגים חדשים של אובייקטים ללא שינוי מהותי בקוד הקיים.

הפרויקט מדגים גם פולימורפיזם, מאחר שניתן לעבוד מול מחלקות בסיס כגון Person, Hall או Ticket, תוך שימוש באובייקטים מסוגי היורשים השונים. גישה זו מאפשרת כתיבת קוד גמיש וניתן להרחבה.

בנוסף, קיימת דוגמה להורשה מרובה באמצעות המחלקה DVIPHall3, היורשת הן מ- VIPHall והן מ- DHall3.

המערכת כוללת גם קשרים בין אובייקטים, כאשר המחלקה Cinema מרכזת ומנהלת אוספים של עובדים, אורחים, סרטים, אולמות ומשמרות. בנוסף קיימים קשרים רבים בין המחלקות, כגון כרטיס המקושר לסרט, אורח המחזיק כרטיסים ומשמרת המקושרת לעובד.

נקודה נוספת היא השימוש ב־העמסת אופרטורים במספר מחלקות, כגון Movie, Date, Cinema, Hall ו- Ticket.

לסיכום, בחרנו בפרויקט Cinematic משום שהוא מדגים מגוון רחב של עקרונות OOP ויכולות של C++, כולל הורשה, פולימורפיזם, הורשה מרובה, קשרים בין אובייקטים והעמסת אופרטורים. לדעתנו מדובר בפרויקט מורכב ומעניין יותר מבחינה תכנונית, ולכן הוא מתאים במיוחד לניתוח במסגרת קורס תכנות מונחה עצמים.

מדוע לא בחרנו בפרויקט Gym System

לאחר בחינת דיאגרמת המחלקות של פרויקט **Gym System**, ראינו כי מדובר בפרויקט מסודר אשר מיישם עקרונות חשובים של תכנות מונחה עצמים, כגון מחלקות אבסטרקטיות, הורשה, קומפוזיציה והורשה מרובה. עם זאת, החלטנו שלא לבחור בו מכיוון שלדעתנו המודל שלו פחות מגוון ופחות עשיר בהשוואה לפרויקט Cinematic.

מבנה המערכת

בפרויקט Gym System רוב המחלקות מאורגנות סביב המחלקה המרכזית GymSystem אשר מנהלת את המתאמנים, המאמנים, השיעורים והציוד. כתוצאה מכך, רוב הקשרים במערכת מתרכזים סביב מחלקה אחת מרכזית, ופחות קיימת חיבור ישירה בין סוגים רבים ושונים של אובייקטים.

היררכיות הירושה

המערכת כוללת מספר היררכיות ירושה, כגון Person שממנה יורשים Member ו־ Trainer וכן Subscription שממנה יורשים MonthlySubscription ו־ AnnualSubscription קיימת גם היררכיה של שיעורים באמצעות ClassSession, PrivateSession ו־ GroupSession למרות זאת, היררכיות אלו קצרות יחסית וממוקדות בתחום פונקציונלי אחד, ולכן לדעתנו הן מציגות מגוון מצומצם יותר של תרחישי OOP .

הרחבת המערכת

מרבית ההרחבות האפשריות בפרויקט הן הרחבות של רכיבים קיימים, למשל הוספת סוג מנוי חדש, סוג שיעור חדש או סוג ציוד חדש. לעומת זאת, בפרויקט Cinematic קיימות מספר משפחות מחלקות שונות לחלוטין (אנשים, אולמות, כרטיסים, סרטים, משמרות ועוד), ולכן קיימות יותר אפשרויות להרחבת המערכת וליצירת אינטראקציות חדשות בין אובייקטים.

סיכום

למרות ש־ Gym System הוא פרויקט טוב, הרגשנו כי פרויקט Cinematic מציג מערכת עשירה ומגוונת יותר מבחינת מבנה המחלקות, סוגי האובייקטים והקשרים ביניהם. לכן בחרנו להתמקד בו, משום שהוא מאפשר להדגים בצורה רחבה יותר את עקרונות התכנות המונחה עצמים ואת היכולות המתקדמות של ++C שנלמדו בקורס.

מסמך שינויים ותיקונים בפרויקט:

1. טבלת סיכום השינויים

מס'	קבצים	תיאור קצר
1	Hall.h, Hall3D.h, VIPHall.h Hall3DVIP.h	תיקון const עבור הסרט שמוקרן באולם
2	Person.h	מחיקת העתקה במחלקת Person
3	Customer.h	מחיקת העתקה במחלקת Customer
4	Employee.h	מחיקת העתקה במחלקת Employee
5	Guest.h	מחיקת העתקה במחלקת Guest
6	Reviewer.h	מחיקת העתקה במחלקת Reviewer
7	Cinema.h	מחיקת העתקה במחלקת Cinema
8	Hall.h, Hall3D.h, VIPHall.h Hall3DVIP.h	מחיקת copy constructors במשפחת Hall
9	Movie.h	הוספת move constructor למחלקת Movie
10	Hall3D.h	הוספת addGlasses למחלקת Hall3D
11	Ticket.h, Ticket.cpp, VIPTicket.h VIPTicket.cpp, main.cpp	קישור Ticket לאולם ולמושב כבר בבנאי
12	Ticket.h, Ticket.cpp	בדיקת זמינות מושב והוספת isSeatFree
13	Ticket.h, Ticket.cpp, VIPTicket.h VIPTicket.cpp, main.cpp	פישוט בנאי Ticket, בדיקת טווח מושב והסרת include מיותר
14	Ticket.h, Ticket.cpp	הסרת set3D מ-Ticket
15	Hall.h, Hall.cpp, Hall3D.h, Hall3D.cpp Ticket.h, Ticket.cpp, VIPTicket.h VIPTicket.cpp, main.cpp	החלפת Ticket::is3D במתודה פולימורפית Hall::isHall3D()
16	Ticket.h, Ticket.cpp	הסרת movieRef מיותר מ-Ticket

17	Hall.h, Hall.cpp, Hall3D.h	ניהול משקפי 3D באמצעות פולימורפיזם
18	main.cpp	שיפור sellTicket: בחירת אולם אוטומטית, ניהול משקפיים ואפשרויות הדפסה
19	Cinema.cpp, main.cpp	מעבר מהצגת אינדקסים מ־0 להצגה מ־1
20	main.cpp	סינון סוגי אולם לפי סרט 3D ב־ addHall

פירוט השינויים שבוצעו

1. תיקון const עבור הסרט שמוקרן באולם

קבצים	Hall.h, Hall3D.h, VIPHall.h, Hall3DVIP.h
מה שונה בפועל	השדה currentMovie והפרמטרים בבנאים שונו מ־ &Movie ל־const Movie.
למה	המתודה const getCurrentMovie() כבר מחזירה &const Movie ולכן גם השדה צריך לשקף זאת. השינוי מונע שינוי לא רצוי של הסרט דרך Hall, מתאים לדגשי const, ומבהיר שהאולם מצביע על סרט קיים אך אינו בעלים שלו ואינו משנה אותו.

2. מחיקת העתקה במחלקת Person

קבצים	Person.h
מה שונה בפועל	ה־copy constructor וה־operator= הוגדרו כ־delete.
למה	ל־Person יש מזהה ייחודי, ולכן שכפול אדם אינו נכון מבחינה סמנטית. מחיקת ההעתקה במחלקת הבסיס מונעת העתקה גם במחלקות היורשות ומונעת טעויות זיכרון או כפילות זהויות.

3. מחיקת העתקה במחלקת Customer

קבצים	Customer.h
מה שונה בפועל	ה־copy constructor וה־operator= הוגדרו כ־delete במקום הצהרה רגילה.
למה	לא ניתן לשכפל לקוח במערכת, משום שלכל Customer יש מזהה ייחודי שמייצג אדם/לקוח ספציפי. לכן מחקנו את האפשרות להשתמש ב־copy constructor וב־operator= עבור Customer כדי למנוע שכפול לקוחות כבר בזמן הקומפילציה.

4. מחיקת העתקה במחלקת Employee

קבצים	Employee.h
מה שונה בפועל	ה־copy constructor וה־operator= הוגדרו כ־delete; ה־destructor נשאר override.
למה	Employee מייצג עובד עם מזהה ייחודי, תאריך לידה ושכר. לכן העתקה אינה רצויה.

5. מחיקת העתקה במחלקת Guest

קבצים	Guest.h
מה שונה בפועל	ה־copy constructor וה־operator= הוגדרו כ־delete.
למה	Guest הוא Person עם זהות ייחודית, ולכן אין היגיון ליצור עותק זהה. המחיקה גם מייתרת צורך במנגנון clone עבור כרטיסים ומפשטת את המערכת.

6. מחיקת העתקה במחלקת Reviewer

קבצים	Reviewer.h
מה שונה בפועל	ה־copy constructor וה־operator= הוגדרו כ־delete; ה־destructor נשאר כי המחלקה מחזיקה publicationName דינמי.
למה	גם Reviewer הוא Person עם מזהה ייחודי ולכן אין לאפשר העתקה.

7. מחיקת העתקה במחלקת Cinema

קבצים	Cinema.h
מה שונה בפועל	ה־copy constructor וה־operator= הוגדרו כ־delete.
למה	Cinema מחזיקה מערכים של עובדים, אורחים, אולמות, סרטים ומשמרות־Shift. מאחר שהמערכת אינה מעתיקה Cinema בפועל, מחיקת ההעתקה בטוחה, בנה ומונעת API מטעה.

8. מחיקת copy constructors במשפחת Hall

קבצים	Hall.h, Hall3D.h, VIPHall.h, Hall3DVIP.h
מה שונה בפועל	ה־copy constructors בכל משפחת האולמות הוגדרו כ־delete.
למה	Cinema אינו מועתק, ולכן גם אולמות אינם אמורים להיות מועתקים. המחיקה מונעת שימוש שגוי.

9. הוספת move constructor למחלקת Movie

קבצים	Movie.h
מה שונה בפועל	נוסף (Movie&& other) Movie לצד ה־copy constructor ו־operator= הקיימים.
למה	Movie היא מחלקת ערך שניתן להעתיק, ומשפרת יעילות כאשר נוצר אובייקט זמני.

10. הוספת addGlasses למחלקת Hall3D

קבצים	Hall3D.h
מה שונה בפועל	נוספה המתודה addGlasses(int amount).
למה	addGlasses מאפשרת להוסיף או להחזיר משקפיים בצורה טבעית בלי לחשוף את הלוגיקה לקוד החיצוני.

11. קישור Ticket לאולם ולמושב כבר בבנאי

קבצים	Ticket.h, Ticket.cpp, VIPTicket.h, VIPTicket.cpp, main.cpp
מה שונה בפועל	Ticket התחיל להחזיק hallRef ו־seatNumber, והבנאי מקבל אולם ומספר מושב. כך הכרטיס מייצג מושב מסוים באולם מסוים.
למה	כרטיס אמיתי חייב להיות קשור לאולם ולמושב, במיוחד כאשר אותו סרט יכול להיות מוקרן בכמה אולמות. השינוי מעביר את האחריות למקום המתאים ומונע יצירת כרטיס שאינו מצביע על מושב ממשי.

12. בדיקת זמינות מושב והוספת isSeatFree

קבצים	Ticket.h, Ticket.cpp
מה שונה בפועל	נוספה בדיקה בבנאי שלא ניתן ליצור Ticket למושב תפוס, ונוספה המתודה isSeatFree().
למה	הבדיקה בבנאי היא שכבת הגנה שמונעת עקיפה של main ויצירת מצב לא תקין. isSeatFree מסתירה את החישוב 1 - seatNumber מהמשתמש ומרכזת את הלוגיקה בתוך Ticket.

13. פישוט בנאי Ticket, בדיקת טווח מושב והסרת include מיותר

קבצים	Ticket.h, Ticket.cpp, VIPTicket.h, VIPTicket.cpp, main.cpp
מה שונה בפועל	הוסר הפרמטר Movie מהבנאי כי הסרט נגזר מהאולם; נוספה בדיקת טווח 1 עד Hall::NUM_SEATS; הוסר include כפול של Movie.h.
למה	הפישוט מונע כפילות: האולם קובע איזה סרט מוקרן. בדיקת הטווח מונעת גישה מחוץ למערך המושבים, והסרת include מיותרת מקטינה תלות בין קבצים.

14. הסרת setIs3D מ־Ticket

קבצים	Ticket.h, Ticket.cpp
מה שונה בפועל	המתודה setIs3D(bool flag) הוסרה.
למה	כרטיס שכבר נמכר לא אמור לשנות את סוגו מ־2D ל־3D או להפך. השארת setter הייתה מאפשרת לשנות מחיר והתנהגות אחרי המכירה, ולכן ההסרה משפרת את אמינות המודל.

15. החלפת Ticket::is3D במתודה פולימורפית Hall::isHall3D()

קבצים	Hall.h, Hall.cpp, Hall3D.h, Hall3D.cpp, Ticket.h, Ticket.cpp, VIPTicket.h, VIPTicket.cpp, main.cpp
מה שונה בפועל	הוסר השדה is3D מ־Ticket. במקום זאת נוספה מתודה וירטואלית isHall3D() ב־Hall שמחזרת false בברירת מחדל ו־true ב־Hall3D.
למה	3D הוא מאפיין של אולם/הקרנה ולא של כרטיס בודד. שימוש בפולימורפיזם מונע מצב שבו Ticket אומר שהוא 3D אבל האולם אינו 3D, ומבטל שדה כפול שעלול לצאת מסבכון.

16. הסרת movieRef מיותר מ־Ticket

קבצים	Ticket.h, Ticket.cpp
מה שונה בפועל	הוסר השדה movieRef &const Movie, והמתודה getMovie() מחזירה כעת hallRef.getCurrentMovie().
למה	שמירת movieRef וגם hallRef יצרה כפילות מידע. יש הפנייה לתוך מחלקת אולם ומתוך אולם מגיעים למחלקת סרט ולא צריך הפנייה לשניהם במחלקה.

17. ניהול משקפי 3D באמצעות פולימורפיזם

קבצים	Hall.h, Hall.cpp, Hall3D.h
מה שונה בפועל	נוספו ב־Hall מתודות וירטואליות addGlasses(), useGlass(), getGlassesCount() עם מימוש ברירת מחדל; Hall3D דורסת אותן.
למה	ב־sellTicket עובדים עם Hall*. בלי מתודות וירטואליות היה צריך לבצע dynamic_cast או בדיקות טיפוס מפורשות.

18. שיפור sellTicket: בחירת אולם אוטומטית, ניהול משקפיים ואפשרויות הדפסה

קבצים	main.cpp
מה שונה בפועל	המשתמש בוחר אורח, סרט, מושב וסוג כרטיס; המערכת מוצאת אוטומטית אולם מתאים. נוספה בדיקת משקפיים, הפחתת משקף בעת מכירת כרטיס 3D, rollback במקרה חריגה, והוספת אפשרויות הדפסה לעובדים, אולמות, סרטים ומשמרות.
למה	הבחירה האוטומטית מונעת מהמשתמש להיחשף לפרטי מימוש, בדיקת המשקפיים מונעת מכירת כרטיס 3D ללא משקפיים, ו־rollback שומר על מצב עקבי אם addTicket נכשל. אפשרויות ההדפסה מנגישות פונקציות שכבר קיימות ב־Cinema.

19. מעבר מהצגת אינדקסים מ־0 להצגה מ־1

קבצים	Cinema.cpp, main.cpp
מה שונה בפועל	רשימות מוצגות למשתמש החל מ־1 במקום 0, והקלט מומר פנימית באמצעות 1 - idx לפני גישה למערך.
למה	ממשק משתמש תפריטי מיועד למשתמשים ולא למתכנתים.

20. סינון סוגי אולם לפי סרט 3D ב־addHall

קבצים	main.cpp
מה שונה בפועל	כאשר הסרט הוא 3D מוצגות רק אפשרויות אולם 3D או 3D VIP; כאשר הסרט אינו 3D מוצגות רק אפשרויות Regular או VIP.
למה	הסינון מונע טעויות קלט: סרט 3D צריך אולם 3D, וסרט רגיל אינו צריך אולם 3D. כך נשמר אינווריאנט עסקי כבר בשכבת הממשק בלי להעמיס בדיקות מיותרות במחלקות.